

Handling Unhealthy Patroni Replica

Overview

This runbook goal is to guide you on the evaluation of one or more unhealthy Patroni replica node that could be causing serious application response time impact, and the steps necessary to remove the unhealthy replica if necessary.

Use this runbook as a guidance on how to diagnose if the Replica is considered Unhealthy and how to safely remove a random node from the Patroni cluster, which is not the same process as scaling down the cluster (that only removes the last nodes of the cluster).

Pre-requisite

- Patroni This runbook assumes that you know what Patroni is, what and how we use it for and possible consequences that might come up if we do not approach this operation carefully. This is not to scare you away, but in the worst case: Patroni going down means we will lose our ability to preserve HA (High Availability) on Postgres. Postgres not being HA means if there is an issue with the primary node Postgres wouldn't be able to do a failover and GitLab would shut down to the world. Thus, this runbook assumes you know this ahead of time before you execute this runbook.
- Chef You are also expected to know what Chef is, how we use it in production, what it manages and how we stop/start chef-client across our hosts.
- Terraform You are expected to know what Terraform is, how we use it and how we make change safely (`tf plan` first).

Scope

This runbook is intended only for one or more **read** replica node(s) of Patroni cluster.

Mental Model

There was an incident but you should not panic, take a deep breath before moving into the steps of this runbook.

Let's build a mental model of what all are at play before you remove a random node from the Patroni cluster.

- We have several Patroni clusters up and running in production
- Some of the replica nodes are taking read requests and processing them, but one or more could be facing issues

- The fact that we have a cluster, it means the cluster might decide to promote any replica to primary (can be the target replica node you are trying to remove)
- There is chef-client running regularly to enforce consistency
- The cluster size is Terraform'd and defined in its respective environment repository

What this means is that we need to be aware of and think of:

- Evaluate which replicas are unhealthy and if is necessary to add more replicas to handle the workload
- Prevent the target replica node from getting promoted to primary
- Stop chef-client so that any change we make to the replica node and patroni doesn't get overwritten
- Take the node out of loadbalancing to drain all connections and then take the replica node out of the cluster
- Safely shutdown and destroy the node
- Let Terraform replace the instance
- Initialize the Patroni service in the replaced instance to re-build it back as a PostgreSQL Replica

Diagnose

Node availability

The main reason that an instance is unhealthy is if it's considered unavailable.

The 3 evidences that point if a Patroni instance is unavailable are:

- If you can't log/ssh into the node;
- If the Thanos/Graphana metrics are missing for the instance - <https://dashboards.gitlab.net/d/bd2Kl9Imk/host-stats>;
- Execute `gitlab-patronictl list` from any other node in the cluster and check if the instance is not listed;

Beside unavailability, a Patroni instance can be considered available but unhealthy for other reasons like Replication Lagging and Resource Contention.

Replication Lagging

If just one or a few Replicas are lagging in relation with the Primary/Writer node there is a great chance that the issue is on the Replica side, so the first evidence of an available but unhealthy replica is replication lag.

Note 1: lag spikes of a few seconds are common; up to `max_standby_stream_delay` (default to 30 seconds) are allowed;

Note 2: if the lag is building up in all nodes, it's more likely that the issue is related with the Writer or the workload and not the case of an unhealthy replica;

- Execute `gitlab-patronictl list` to get the amount of Lag, in MBytes, for each Replica

For example the following output show aprox. 19 GB (19382 MB) of lag in the `patroni-08-db-gstg` host

```
# gitlab-patronictl list
+ Cluster: pg12-ha-cluster-stg (6951753467583460143) -----+-----+-----+
| Member                                     | Host          | Role  | State  | TL | Lag in MB | Tags |
+-----+-----+-----+-----+-----+-----+-----+
| patroni-01-db-gstg.c.gitlab-staging-1.internal | 10.224.29.101 | Leader | running |    |          |      |
+-----+-----+-----+-----+-----+-----+-----+
| patroni-02-db-gstg.c.gitlab-staging-1.internal | 10.224.29.102 | Replica | running |    |          |      |
+-----+-----+-----+-----+-----+-----+-----+
| ...                                         |                |       |        |    |          |      |
+-----+-----+-----+-----+-----+-----+-----+
| patroni-07-db-gstg.c.gitlab-staging-1.internal | 10.224.29.107 | Replica | running |    |          |      |
+-----+-----+-----+-----+-----+-----+-----+
| patroni-08-db-gstg.c.gitlab-staging-1.internal | 10.224.29.108 | Replica | running |    |          |      |
+-----+-----+-----+-----+-----+-----+-----+
```

- Or you can look into the `Lag time` or `Lag size` Graphana dashboards, or `pg_replication_lag` or `pg_stat_replication_pg_wal_lsn_diff` Thanos metrics.

Resource Contention

If there is intense resource contention a resource can become unhealthy and get stuck/unavailable, check for:

- CPU usage stuck close to 100%
- Disk Metrics (eg. I/O wait per operation)
- Look for Stuck I/O and Disk Failure in syslog
- Memory Swapping (ocasional swapping can happen, intense swapping can cause PostgreSQL to hang)
- OOM Kill messages in syslog or VM serial console, see OOM and Memory Pressure

OOM and Memory Pressure Our production database servers currently have an abundance of free memory. This is necessary to have enough headroom for peeks and also to keep a significant portion of the available memory unallocated to be used by the file system cache. Without large free memory for the fs cache we will start to see more disk read load, which in an worst case scenario could saturate and slow down the application. This should trigger memory saturation alerts as well as Apdex degradation alerts.

If memory pressure continues to a point where not enough memory is available to keep operating the Linux Kernel will start the out of memory killer. In

general, it is not advised for PostgreSQL to over provision memory, but due to third party software running systems it was decided to not disable it globally, but only for the cgroup PostgreSQL and Patroni are running in. Details can be found here [Our production database servers currently have an abundance of free memory.](#)

Should the OOM kiler become active it will start killing processes based on their OOM score. For us it will be processes outside the PostgreSQL cgroup consuming significant memory. This will most likely trigger exporters or log collectors, which should trigger additional alerts.

Investigation Under no circumstances should you ever terminate PostgreSQL process with SIGKILL aka. `kill -9`! This will trigger a complete shutdown of the affected PostgreSQL instance, more details here [PostgreSQL, memory and the cloud.](#)

It should be investigated which processes are consuming problematic amounts of memory. If PostgreSQL's processes are involved it should be checked if they belong to a long-running transaction. Use the following command to find the longest running queries on the system.

```
gitlab-psql -c "SELECT * FROM pg_stat_activity ORDER BY query_start ASC LIMIT 10;"
```

Mitigation Once identified, a problematic query can be terminated *safely* via `pg_terminate_backend ( pid, ... )`.

```
gitlab-psql -c "SELECT pg_terminate_backend(xxxxx);"
```

Should the memory consumption not be the result of a few queries consuming an excessive amount of memory, but caused by many smaller queries it needs to be investigated where the consumption is coming from. Involve the database team to get inside and help to analyze the query plans.

A first go-to mitigation should be to add more replicas to spread the queries over more machines, reducing the memory pressure per system.

Draining Workload from the Unhealthy Patroni replica

Preparation

- You should do this activity in a CR (thus, allowing you to practice all of it in staging first)
- Make sure the replica you are trying to remove is NOT the primary, by running `gitlab-patronictl list` on a patroni node
- Pull up the Host Stats Grafana dashboard and switch to the target replica host to be removed. This will help you monitor the host.

Step 1 - Stop chef-client

1. On the replica node run: `sudo chef-client-disable "Removing patroni node: Ref issue prod#xyz"`

Step 2 - Take the replicate node out of load balancing

If clients are connecting to replicas by means of service discovery (as opposed to hard-coded list of hosts), you can remove a replica from the list of hosts used by the clients by tagging it as not suitable for failing over (`nofailover: true`) and load balancing (`noloadbalance: true`). (If clients are configured with `replica.patroni.service.consul`. DNS record look at this legacy method)

1. Add a tags section to `/var/opt/gitlab/patroni/patroni.yml` on the node:

tags:

```
nofailover: true
noloadbalance: true
```

1. Reload Patroni

```
sudo systemctl reload patroni
```

1. Check that Patroni host now is no longer considered for failover nor load-balance

```
sudo gitlab-patronictl list
```

2. Test the efficacy of that reload by checking for the node name in the list of replicas:

```
dig @127.0.0.1 -p 8600 db-replica.service.consul. SRV
```

If the name is absent, then the reload worked.

1. Wait until all client connections are drained from the replica (it depends on the interval value set for the clients), use this command to track number of client connections:

```
for c in /usr/local/bin/pgb-console*; do $c -c 'SHOW CLIENTS;' | grep gitlabhq_production |
```

It can take a few minutes until all connections are gone. If there are still a few connections on pgbouncers after 5m you can check if there are actually any active connections in the DB (should be 0 most of the time):

```
gitlab-psql -qc \  
"select count(*) from pg_stat_activity  
where backend_type = 'client backend'  
and state <> 'idle'  
and pid <> pg_backend_pid()  
and datname <> 'postgres'"
```

You can see an example of taking a node out of service in this issue.

Step 3 - Decide if you will remove the node or wait for it to recover

This is a critical decision because replacing a Patroni Replica can take a couple of hours, but letting it to recover without intervention can take up to several days depending on lag size and write workload.

If you decide to relpace the unhealthy replica proceed to the next chapter.

Removing an unhealthy replica from the Patroni cluster

IMPORTANT: make sure that the connections that the workload is drained from the unhealthy replica (link to previous chapter)

Step 1 - Stop patroni service on the node

Now it is safe to stop the patroni service on this node. This will also stop postgres and thus terminate all remaining db connections if there are still some. With the patroni service stopped, you should see this node vanish from the cluster after a while when you run `gitlab-patronictl list` on any of the other nodes.

We have alerts that fire when patroni is deemed to be down. Since this is an intentional change - either silence the alarm in advance and/or give a heads up to the EOC (by messaging @sre-oncall at #infrastructure-lounge Slack channel).

1. Stop the patroni service on the unhealthy node

```
sudo systemctl stop patroni
sudo systemctl disable patroni.service
```

1. Check that patroni service is stopped in the host

```
sudo gitlab-patronictl list
```

Step 2 - Shutdown the node

1. Find the instance name in GCloud
2. You can use `gcloud compute instances list | grep <name>` to search for the instances, or get it using GCP console <https://console.cloud.google.com/compute/instances>;
3. Define the `INSTANCE_NAME=<name>` variable with the proper instance name
4. Stop the VM

```
gcloud compute instances stop $INSTANCE_NAME
```

Step 3 - Delete the VM and disks

1. List the VM Disks

Execute the following script lines:

```
...
```

```
IFS=","
```

```
echo "## List disks:"
```

```
gcloud compute instances describe $INSTANCE_NAME --format="value(disks.source.list())"
```

```
echo "## Delete Disks Commands - TAKE NOTE of them:"
```

```
for disk in $(gcloud compute instances describe $INSTANCE_NAME --format="value(disks.source.list())" | sed -n 's/disk: //p'); do
```

```
do
    echo "Run: gcloud compute disks delete $disk --zone <zone_name>"
```

```
done
```

```
...
```

1. Delete the VM

- Execute the following command:

```
gcloud compute instances delete $INSTANCE_NAME
```

- Take note of the zone where the instance is hosted, you will need to delete the instance disks;

1. Delete the Disks

- Execute the delete disks command listed in the “List VM Disks” step. You would need to delete only the **data** disk and the **log** disk of the Patroni node as the instance disk is automatically deleted with the instance.

- If you didn't took note of the delete disk commands above, execute the following commands to delete the **data** and **log** disks:

```
gcloud compute disks delete $INSTANCE_NAME-data --zone <zone_name>
```

```
gcloud compute disks delete $INSTANCE_NAME-log --zone <zone_name>
```

1. Confirm that Compute instances and disks were deleted in the GCP console:

- <https://console.cloud.google.com/compute/instances>
- <https://console.cloud.google.com/compute/disks>

1. Check if the instance nodes still exist in Chef and delete them if necessary

- Execute the following command:

```
ENVIRONMENT=<enter the environment, eg. gstg or gprd>
```

```
cd <your chef-repo directory>
```

```
knife node list | grep $ENVIRONMENT | grep $INSTANCE_NAME
```

```
knife client list | grep $ENVIRONMENT | grep $INSTANCE_NAME
```

- If the node still is listed delete the node from Chef server with:

```
knife node delete <NODE_NAME>
knife client delete <NODE_NAME>
```

Replacing the removed replica

Step 1 - Take a Disk Snapshot of the backup node, to recreate the replica

1. Find which instance is the database cluster Backup Node
 - GSTG Main: `knife search 'roles:gstg-base-db-patroni-backup-replica AND roles:gstg-base-db-patroni-main' --id-only`
 - GSTG CI: `knife search 'roles:gstg-base-db-patroni-ci-backup-replica AND roles:gstg-base-db-patroni-ci' --id-only`
 - GPRD Main: `knife search 'roles:gprd-base-db-patroni-backup-replica AND roles:gprd-base-db-patroni-v12' --id-only`
 - GPRD CI: `knife search 'roles:gprd-base-db-patroni-ci-backup-replica AND roles:gprd-base-db-patroni-ci' --id-only`
2. Log into the Backup Node and execute a gcs-snapshot:

```
sudo su - gitlab-psql
PATH="/usr/local/sbin:/usr/sbin/::sbin:/usr/local/bin:/usr/bin:/bin:/snap/bin"
/usr/local/bin/gcs-snapshot.sh
```

Step 2 - Recreate the removed node

You can use the following steps to create all or a subset of the patroni CI instances, just depending on how many instances were previously destroyed.

1. Change Terraform environment
 - Execute the following `gcloud` command to get the name of the most recent GCS snapshot from the patroni backup data disk, but **DO NOT SIMPLY COPY/PASTE IT**, set the `--project` and `--filter` accordingly with the environment you are performing the restore:

```
gcloud compute snapshots list --project [gitlab-staging-1|gitlab-production] --limit=1
```
 - Remove the `https://www.googleapis.com/compute/v1/` prefix of the snapshot name
 - For example: `https://www.googleapis.com/compute/v1/projects/gitlab-production/global/` will turn into `projects/gitlab-production/global/snapshots/nukw46z00o90`
 - Add the following line into the proper `patroni-*` module at `main.tf`

```
data_disk_snapshot    = "<snapshot_name>"
data_disk_create_timeout = "180m"
```
1. Check the resources that will be created on the plan


```
tf plan -out resync-tf.plan
```

- Check the Terraform change before applying, TF should create 3 resources for each removed Patroni Replica: 2 disks and 1 VM/instance
 - module.patroni.google_compute_disk.data_disk
 - module.patroni.google_compute_disk.log_disk
 - module.patroni.google_compute_instance.instance_with_attached_disk

1. Re-create the unhealthy patroni node

```
tf apply "resync-tf.plan"
```

1. Check the VM instance Serial port in the GCP console to see if the instance is already initialized and if Chef has finished running, for example:
 - GSTG Main: instance patroni-01-db-gstg/console?port=1&project=gitlab-staging-1
 - GPRD Main: instance patroni-v12-01-db-gprd/console?port=1&project=gitlab-production
 - Or you can execute `gcloud compute instances get-serial-port-output <instance_name>`
2. Start patroni service on the node

```
ssh <node_fqdn> "systemctl enable patroni && systemctl start patroni"
```

Step 3 - Check Patroni, PGBouncer and PostgreSQL

- Login into the node and check if Patroni is running and in sync with Writer/Primary
 - Node patroni status and lag

```
sudo gitlab-patronictl list
```
 - If the node is not in 'running' state with 0 lag, check the postgresql logs:

```
tail -n 1000 -f /var/log/gitlab/postgresql/postgresql.csv
```
- Checking for the node name in the list of replicas in Consul:

```
dig @127.0.0.1 -p 8600 db-replica.service.consul. SRV
```

If the name of the replaced host is in the list it should start receiving connections;
- Check Pgbouncer status:

```
for c in /usr/local/bin/pgb-console*; do $c -c 'SHOW CLIENTS;'; done;
```
- Check PostgreSQL for connected clients:

```
sudo gitlab-psql -qc \  
"select count(*) from pg_stat_activity
```

```
where backend_type = 'client backend'
and pid <> pg_backend_pid()
and datname <> 'postgres'"
```

Reference

Patroni Management Internal Doc.